

Computer Security Capstone

Chapter 4: Access Control

Chi-Yu Li (2025 Spring)
Computer Science Department
National Yang Ming Chiao Tung University

Definition of Computer Security (RFC 4949)

Measures that implement and assure security services in a computer system, particularly those that assure access control service.

Access control: the central element of computer security

Principal Objectives of Computer Security

- Prevent **unauthorized** users from gaining access to resources
- Prevent **legitimate** users from accessing resources in an **unauthorized** manner
- Enable **legitimate** users to access resources in an **authorized** manner

Example Attack Scenarios

- Failures typically lead to

- ❑ Unauthorized info disclosure, modification, or destruction of all data
- ❑ Performing a business function outside the user's limits

- Scenario #1: using unverified data in a SQL call

```
pstmt.setString(1, request.getParameter("acct"));  
ResultSet results = pstmt.executeQuery( );
```

- ❑ Attacker can access any user's account

```
https://example.com/app/accountInfo?acct=notmyacct
```

Example Attack Scenarios (cont.)

- Scenario #2: forcing browsers to target URLs

```
https://example.com/app/getappInfo  
https://example.com/app/admin_getappInfo
```

- Non-admin attacker can access the admin page

Example Hacks due to Broken Access Control

- Cambridge Analytica Scandal (2018)

- ❑ Attacker: Cambridge Analytica (a political consulting company)
- ❑ Attack: Accessing to the personal info of millions of FB users w/o their consent
- ❑ Root cause: Vulnerability in FB's API that allowed third-party developers to access user data
- ❑ Impact: FB faced widespread criticism and was fined \$5 billion by the U.S. Federal Trade Commission

Example Hacks due to Broken Access Control (cont.)

● Target Data Breach (2013)

- ❑ Victim: Target (discount department stores)
- ❑ Attack and impact: A data breach that exposed the credit/debit card info of approximately 40 million customers
- ❑ Root cause: security misconfiguration in Target's payment processing system
 - Network traffic segregation of different departments and systems is not properly implemented
 - Hackers could access the payment processing system through the insecure vendor portal

Outline

- Access Control Principles
- Subjects, Objects, and Access Rights
- Discretionary Access Control
- Example: Unix File Access Control
- Role-Based Access Control
- Attribute-Based Access Control
- Case Study: RBAC System for a Bank

Access Control Context

- Authentication

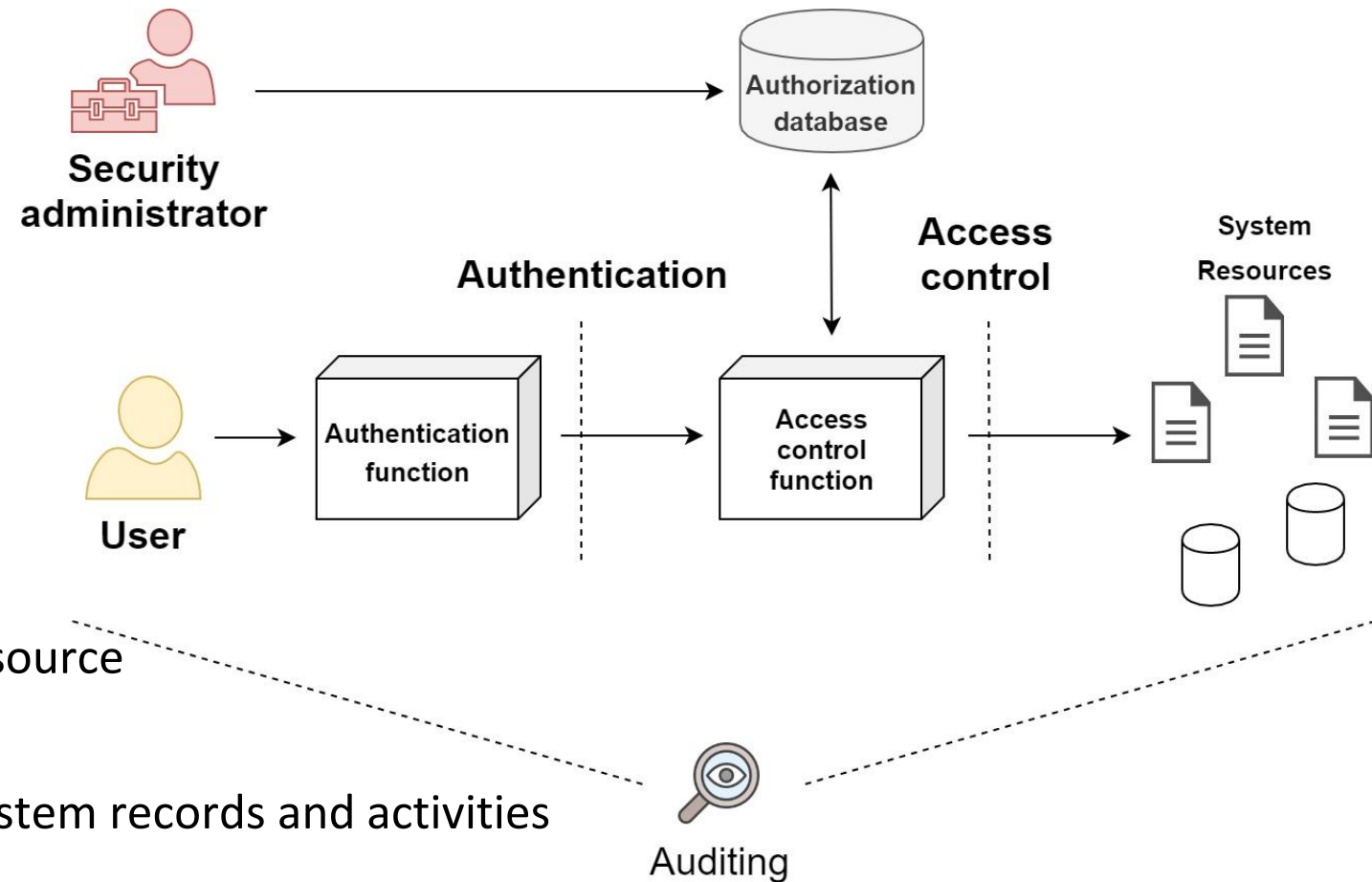
- ▣ Verifying that user/system credentials are valid

- Authorization

- ▣ Granting a right or permission to a system entity to access a system resource

- Audit

- ▣ An independent examination of system records and activities
 - Test adequacy of system controls
 - Ensure compliance with established policy and operational procedures
 - Detect breaches in security
 - Recommend any indicated changes in control, policy and procedures



Access Control Policies

● Discretionary access control (DAC)

- ❑ Based on the requestor's identity
- ❑ Access on rules stating what requestors are (or are not) allowed to do
- ❑ Why discretionary?
 - An entity might have access rights to enable another entity to access some resource

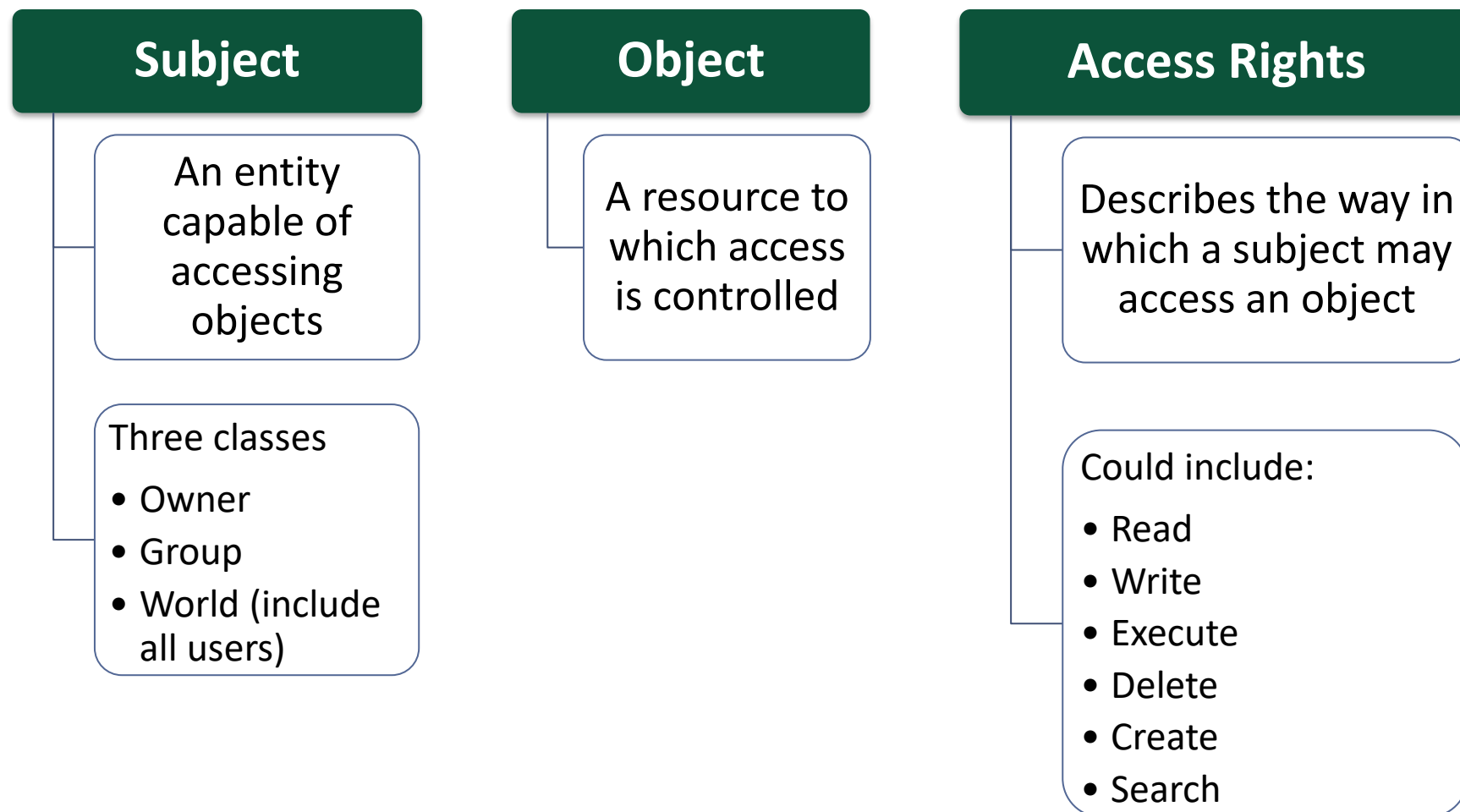
● Mandatory access control (MAC)

- ❑ Based on security clearances of system entities
- ❑ Access on security labels of resources
- ❑ Why mandatory?
 - An entity that has clearance to access a resource may not enable another entity to access that resource

Access Control Policies (Cont.)

- Role-based access control (RBAC)
 - ❑ Based on the users' roles
 - ❑ Access on rules stating what accesses are allowed to given roles
- Attribute-based access control (ABAC)
 - ❑ Based on the users' attributes, the accessed resource, and current environmental conditions

Basic Elements



Discretionary Access Control (DAC)

- A general approach: access matrix

- Subjects vs. Objects
- Each entry: access right

SUBJECTS

OBJECTS

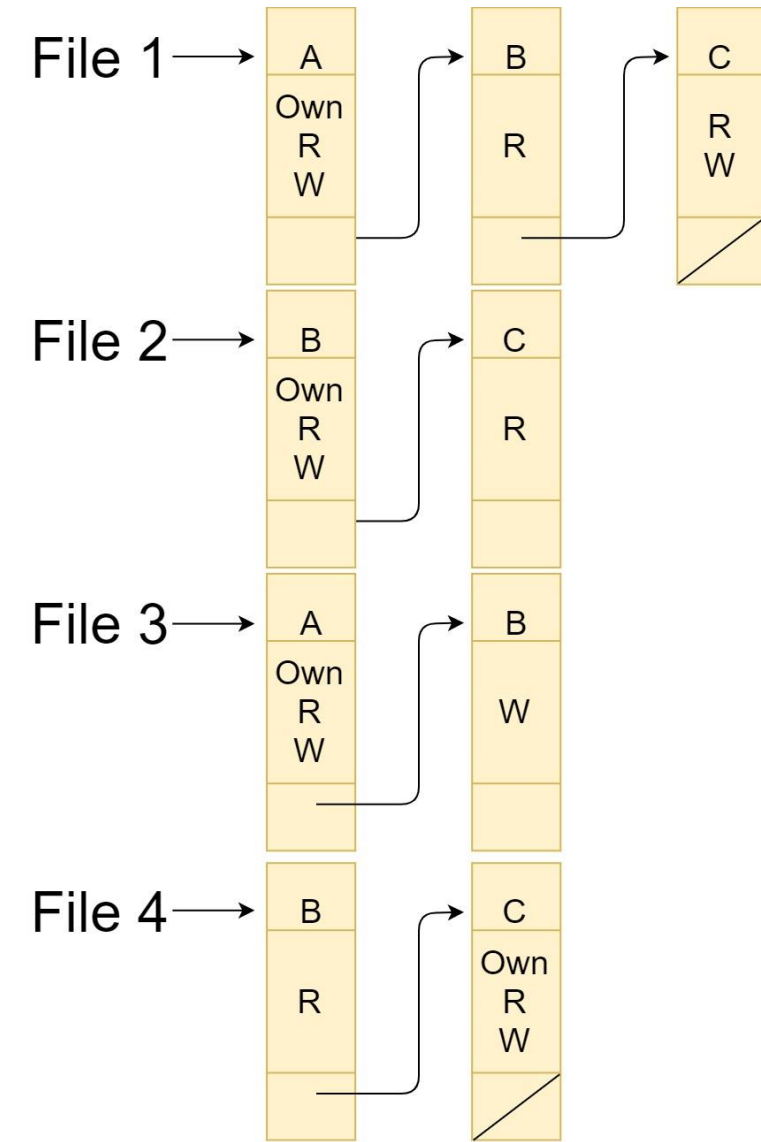
	File 1	File 2	File 3	File 4
User A	Own Read Write		Own Read Write	
User B	Read	Own Read Write	Write	Read
User C	Read Write	Read		Own Read Write

In practice, an access matrix is usually sparse!

Access matrix

Decomposition Method I

- Access control lists (ACL): decomposed by columns (objects)
 - ❑ For each object, an ACL lists users and their permitted access rights
 - ❑ Default set of rights: users that are not explicitly listed
 - ❑ Convenient: determining which subjects have which access rights to a particular resource
 - ❑ Inconvenient: determining the access rights available to a specific user



Decomposition Method II

- **Capability tickets: decomposed by rows (subjects)**

- A capability ticket specifies authorized objects and operations for a particular user

- Convenient/Inconvenient: opposite to ACLs

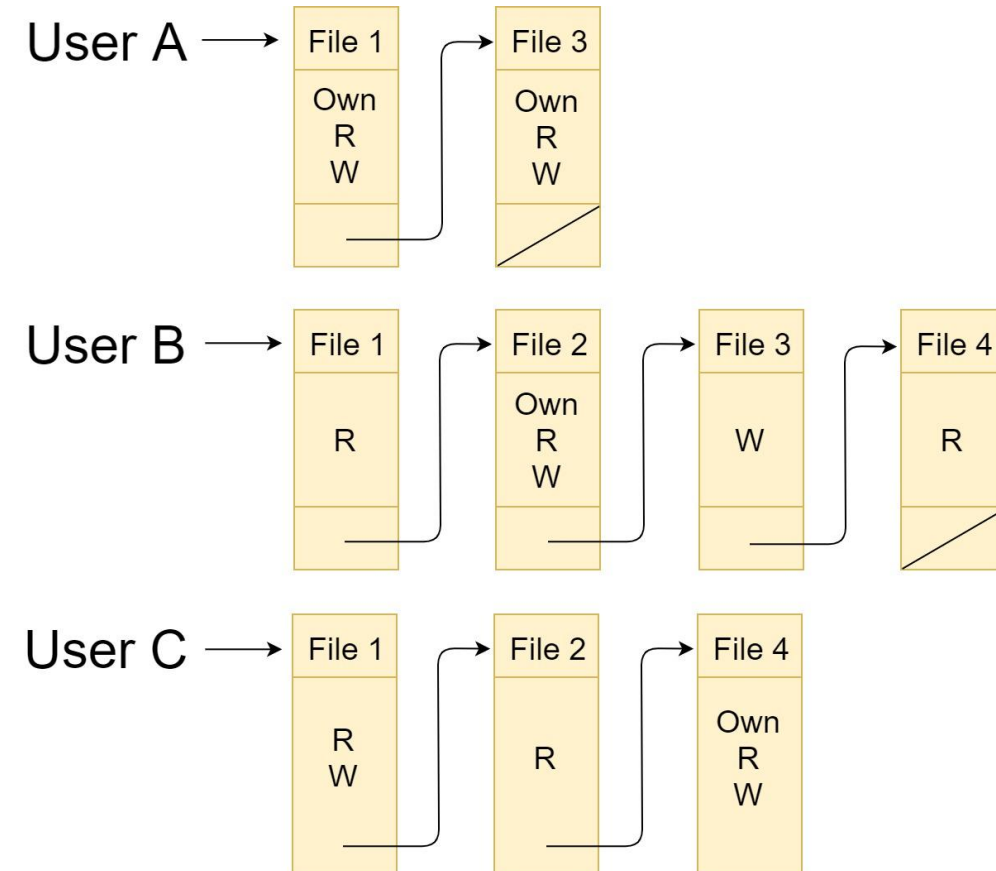
- **Have greater security problem than ACLs. Why?**

- Tickets may be dispersed around the system

- Integrity of the ticket must be protected, guaranteed, and unforgeable

- Two solutions

- OS holds all tickets on behalf of users
 - An unforgeable token in the capability



Another Approach: Authorization Table [SAND94]

- Not sparse and more convenient than either ACLs or capability lists
- A relational database can easily implement an authorization table of this type
- Any drawback?

Subject	Access Mode	Object
A	Own	File 1
A	Read	File 1
A	Write	File 1
A	Own	File 3
A	Read	File 3
A	Write	File 3
B	Read	File 1
B	Own	File 2
B	Read	File 2
B	Write	File 2
B	Write	File 3
B	Read	File 4
C	Read	File 1
C	Write	File 1
C	Read	File 2
C	Own	File 4
C	Read	File 4
C	Write	File 4

A General Access Control Model for DAC

- Three requirements

- ❑ Representing the protection state
- ❑ Enforcing access rights
- ❑ Allowing subjects to alter the protection state in certain ways

- Concepts

- ❑ As usual: a set of subjects, objects, and rules
- ❑ New: protection states

- Protection states

- ❑ Processes: delete, stop (block), and wake up
- ❑ Devices: read/write, operation control, and block/unblock
- ❑ Memory locations or regions: read/write
- ❑ Subjects: grant or delete access rights of objects

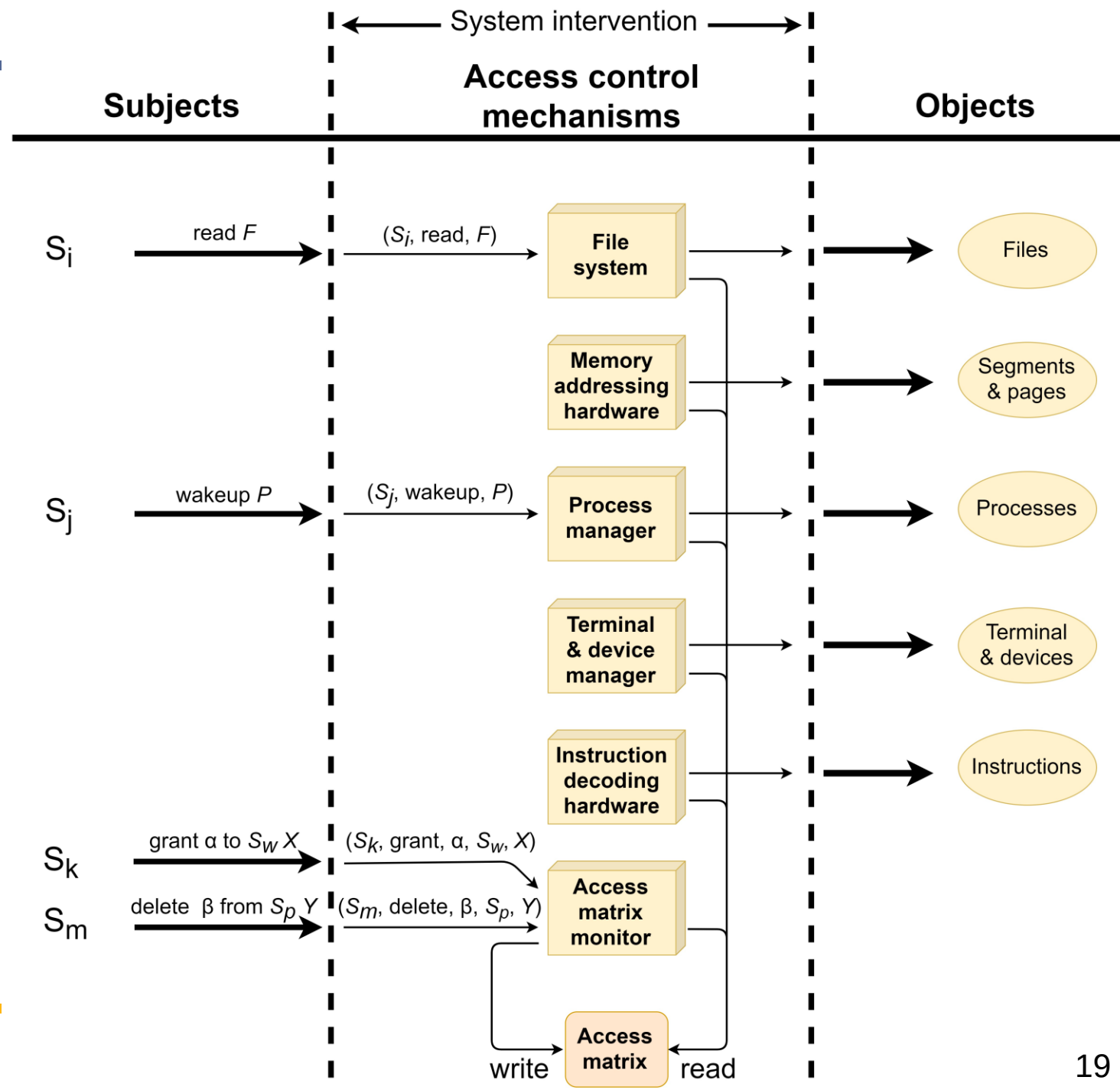
Example: Extended Access Control Matrix

		OBJECTS								
		subjects			files		processes		disk drivers	
		S ₁	S ₂	S ₃	F ₁	F ₂	P ₁	P ₂	D ₁	D ₂
SUBJECTS	S ₁	control	owner	owner control	read *	read owner	wakeup	wakeup	seek	owner
	S ₂		control		write *	execute			owner	seek *
	S ₃			control		write	stop			

* - copy flag set

Example: Access Control Function

- Every access by a subject to an object is mediated by the controller for that object
- Decisions are based on access matrix monitor



More Flexible Model: Protection Domains

- A set of objects together with access rights to those objects
 - e.g., the access matrix
 - A row defines a protection domain
 - Each user has a protection domain → Any processes spawned by the user have access rights of the same domain

Do the processes really need all the access rights?

- Recall security design principles: Least privilege
 - Every process and every user of the system should operate using the least set of privileges necessary to perform the task

Protection Domains (Cont.)

- More general concept: minimize the access rights that any user or process has at any one time
 - e.g., A user: spawns processes with a subset of the access rights of the user
 - Limit the capability of the processes
- Association between a process and a domain can be static or dynamic
 - e.g., A process: a sequence of procedures require different access rights
- One form: distinction mode in many OSes (e.g., UNIX)
 - User mode: certain areas of memory are protected and certain instructions may not be executed
 - Kernel mode

Example: UNIX File Access Control

UNIX files are administered using inodes (index nodes)

- An inode: a control structure with key information for a particular file
- Several file names may be associated with a single inode; inode and file are 1-1 mapping
- File attributes, permissions and control information are stored in the inode
- On the disk there is an inode table, or inode list, that contains the inodes of all the files in the file system
- When a file is opened its inode is brought into main memory and stored in a memory resident inode table

Directories are structured in a hierarchical tree

- May contain files and/or other directories
- Simply a file: contains file names plus pointers to associated inodes

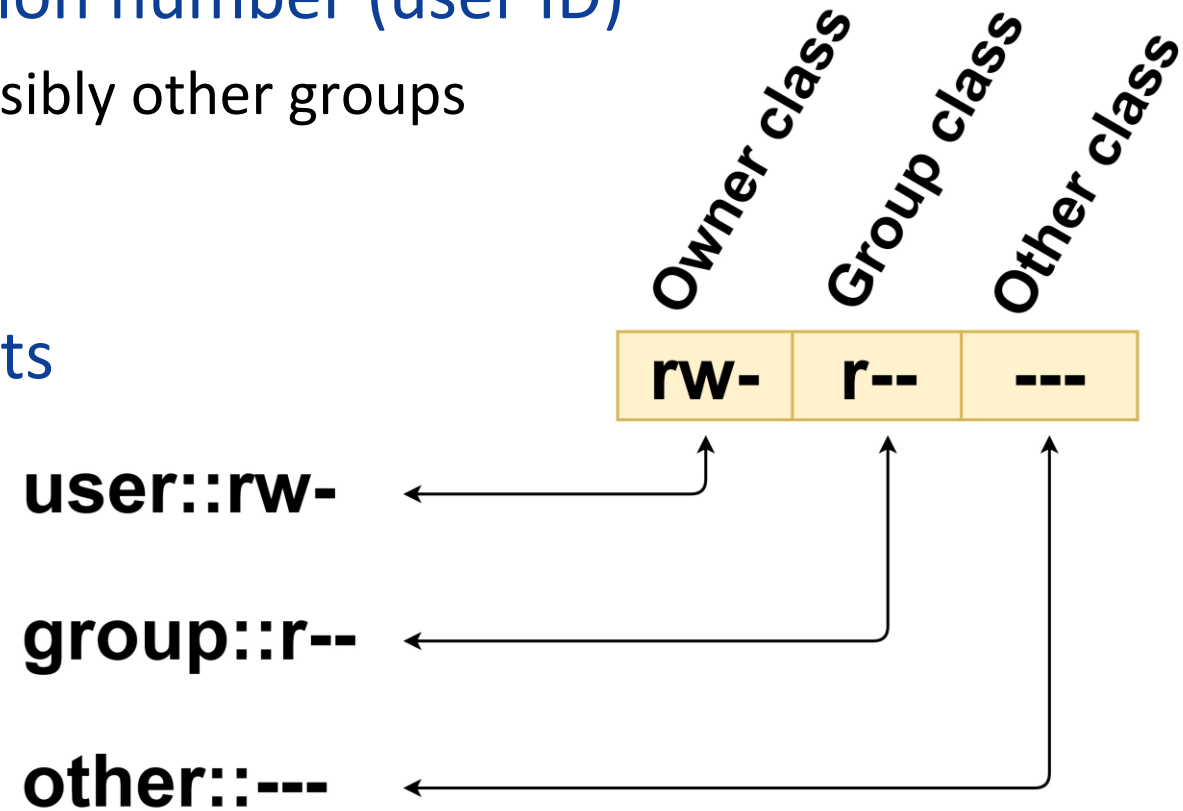
Traditional UNIX File Access Control

- UNIX user: a unique user identification number (user ID)

- ❑ A member of a primary group, and possibly other groups
- ❑ Each group is identified by a group ID

- Each file/directory: 12 protection bits

- ❑ First 9 bits: read, write, execute
- ❑ Last 3 bits: setUID, setGID, and sticky



Traditional UNIX File Access Control (Cont.)

● SetUID/SetGID bits

- ❑ Known as the “effective user ID” and “effective group ID”
- ❑ System temporarily grants a real user with the rights of the file owner/group in addition to the real user’s rights
- ❑ For executable files
 - Only effective while the program is being executed
 - Allows users to run programs with temporarily elevated privileges to perform a specific task
 - e.g., the `ping` command: need access to networking privileges that a normal user cannot access
- ❑ For directories
 - SetGID: newly created files will inherit the group of this directory, rather than the primary group ID of the user who created this file
 - SetUID is ignored
- ❑ Security risk?

Traditional UNIX File Access Control (Cont.)

□ Examples: passwd and ping

setuid: 4
setgid: 2

```
chiyuli@linux1:~ [83x25]
Connection Edit View Window Option Help
[chiyuli@linux1 ~]$ stat -c "%a %U:%G %n" /usr/bin/passwd
4755 root:root /usr/bin/passwd
[chiyuli@linux1 ~]$ stat -c "%a %U:%G %n" /etc/passwd
644 root:root /etc/passwd
[chiyuli@linux1 ~]$ stat -c "%a %U:%G %n" /etc/shadow
0 root:root /etc/shadow
[chiyuli@linux1 ~]$ passwd
Please enter your old LDAP(Linux/FreeBSD) password: █
chiyuli@linux1:~ [83x25]
Connection Edit View Window Option Help
[chiyuli@linux1 ~]$ stat -c "%a %U:%G %n" /bin/ping
755 root:root /bin/ping
[chiyuli@linux1 ~]$ getcap /bin/ping
/bin/ping = cap_net_admin,cap_net_raw+p
[chiyuli@linux1 ~]$ getcap /usr/bin/passwd
[chiyuli@linux1 ~]$ █
```

Traditional UNIX File Access Control (Cont.)

● Sticky bit

- ❑ Files: the system should retain the file contents in memory following execution (no longer used)
- ❑ Directories: only the owner of any file in the directory can rename, move, or delete that file
 - Useful for managing files in shared temporary directories

● Superuser

- ❑ Exempts from the usual file access control constraints
- ❑ Needs great care on the programs owned by and setuid set to “superuser”

Traditional UNIX File Access Control (Cont.)

- What issues does this access scheme have?

- Consider one scenario

- Read access for file X to Users A and B
 - Read access for file Y to Users B and C

- Need at least two user groups

- What if there are a large number of different groupings of users requiring a range of access rights to different files?

- No scalability: unwieldly and difficult to manage

Modern UNIX Access Control: Access Control Lists (ACLs)

- Supported by many modern UNIX-based OSes

- e.g., FreeBSD, OpenBSD, Linux, and Solaris
- Extended ACL vs. minimal ACL (traditional)

- FreeBSD

- Any number of users and groups can be assigned to a file
 - Each with three protection bits
- A file need not have an ACL; may be protected solely by traditional access control
- An additional protection bit: whether the file has an extended ACL

Modern UNIX Access Control (Cont.)

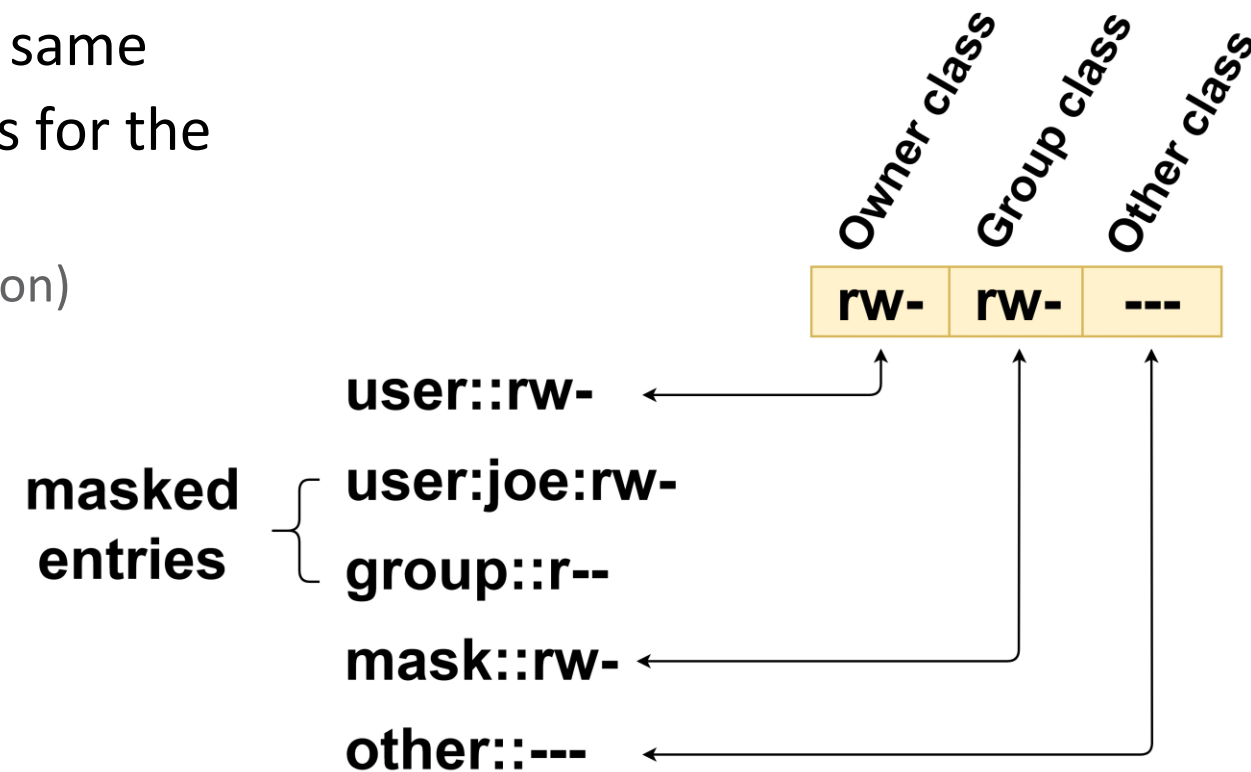
- Extended ACLs are used with the following strategies

- ❑ Owner and other classes remain the same
- ❑ Group class specifies the permissions for the owner group for this file

- Functions as a mask (maximum permission)

- ❑ Additional named users and named groups may be associated with the file

- Each with a 3-bit permission field



Examples

```
[root@linux ~]# setfacl -m u:bob:rwX test
[root@linux ~]# getfacl test
# file: test
# owner: root
# group: root
user::rwx
user:bob:rwX
group::r--
mask::rwx
other::r--
```

Step 1

```
[root@linux ~]# setfacl -m g:cs:rX test
[root@linux ~]# getfacl test
# file: test
# owner: root
# group: root
user::rwx
user:bob:rwX
group::r--
group:cs:rX
mask::rwx
other::r--
```

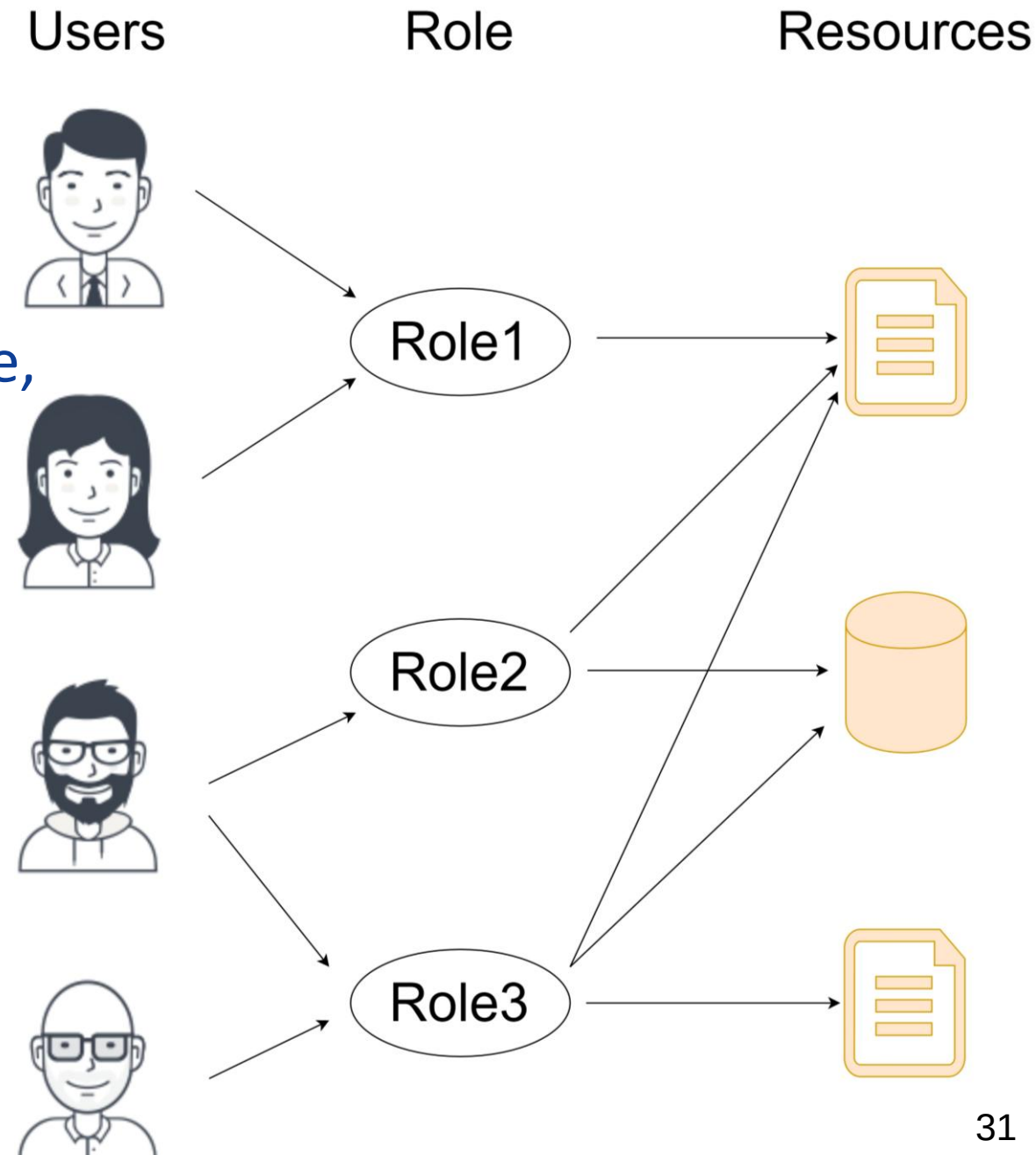
Step 2

```
[root@linux ~]# setfacl -m m:r test
[root@linux ~]# getfacl test
# file: test
# owner: root
# group: root
user::rwx
user:bob:rwX           effective: ?
group::r--
group:cs:rX             effective: ?
mask::r--
other::r--
```

Step 3

Role-based Access Control (RBAC)

- Based on the roles that users assume, instead of their identities
- Widespread commercial use and an area of active research
- Many-to-many relationship
 - ❑ users to roles
 - ❑ roles to resources



Access Control Matrix for RBAC

- RBAC: obeys principle of “least privilege”
 - ▣ Each role contains the minimum set of access rights needed for that role
 - ▣ A user is assigned to a role that enables him or her to perform only what is required for that role

		OBJECTS								
		R ₁	R ₂	R _n	F ₁	F ₁	P ₁	P ₂	D ₁	D ₂
ROLES	R ₁	control	owner	owner control	read *	read owner	wakeup	wakeup	seek	owner
	R ₂		control		write *	execute			owner	seek *
	⋮									
	R _n			control		write	stop			

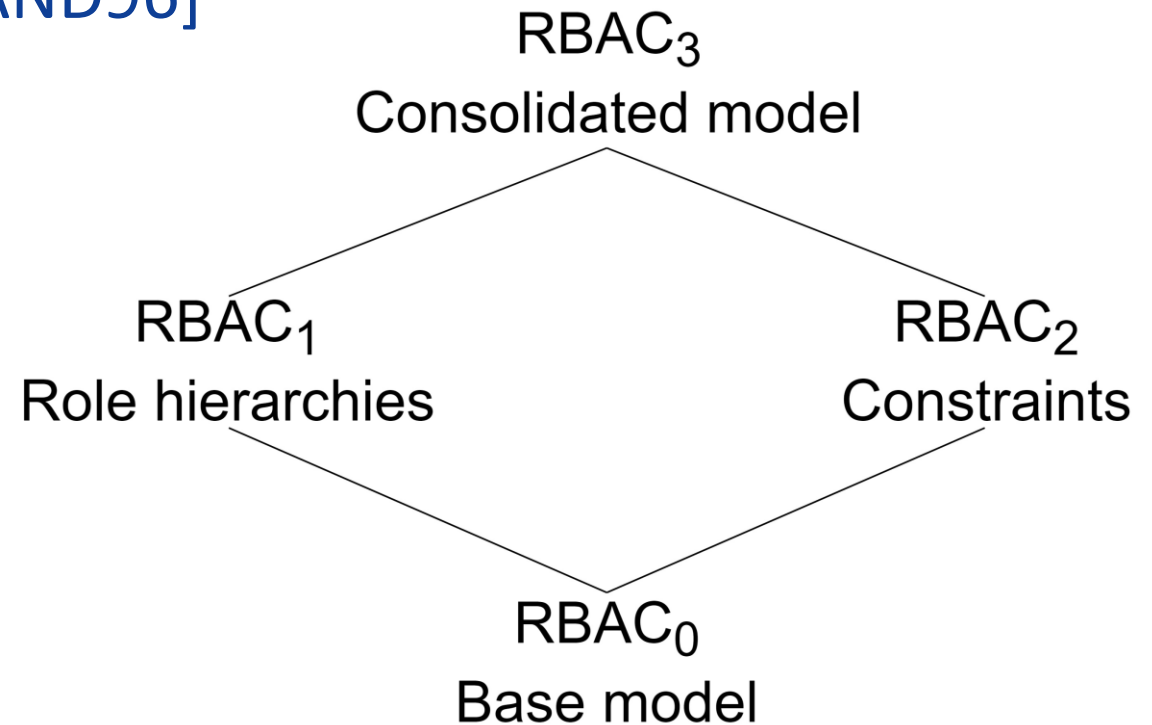
	R ₁	R ₂	⋯	R _n
U ₁	X			
U ₂	X			
U ₃		X		X
U ₄				X
U ₅				X
U ₆				X
⋮				
U _m	X			

RBAC Reference Models

- A family of reference models have been defined as the basis for ongoing standardization efforts [SAND96]

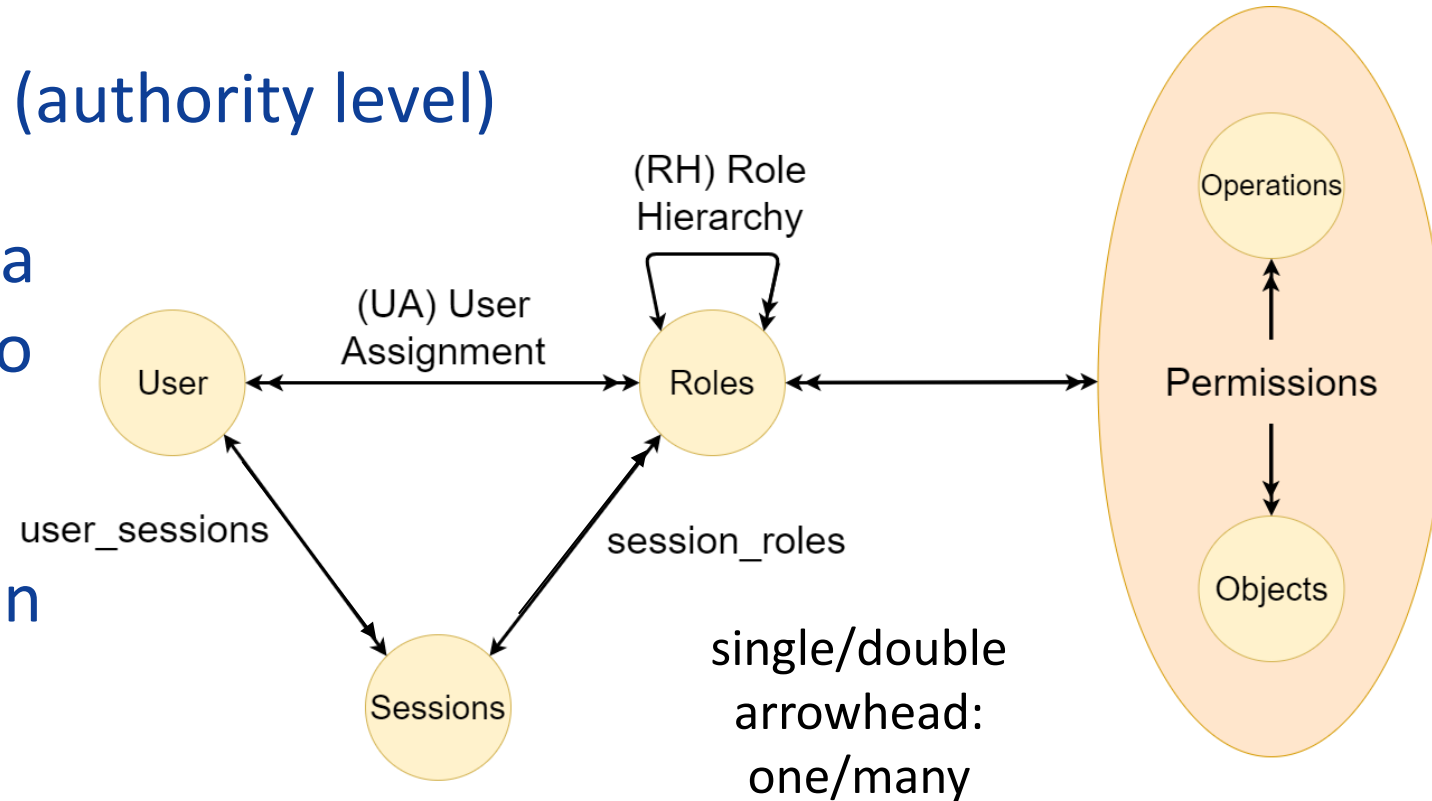
- Four models

- $RBAC_0$: minimum functionality
- $RBAC_1$: $RBAC_0$ + role hierarchies
- $RBAC_2$: $RBAC_0$ + constraints
- $RBAC_3$: $RBAC_0$ + $RBAC_1$ + $RBAC_2$



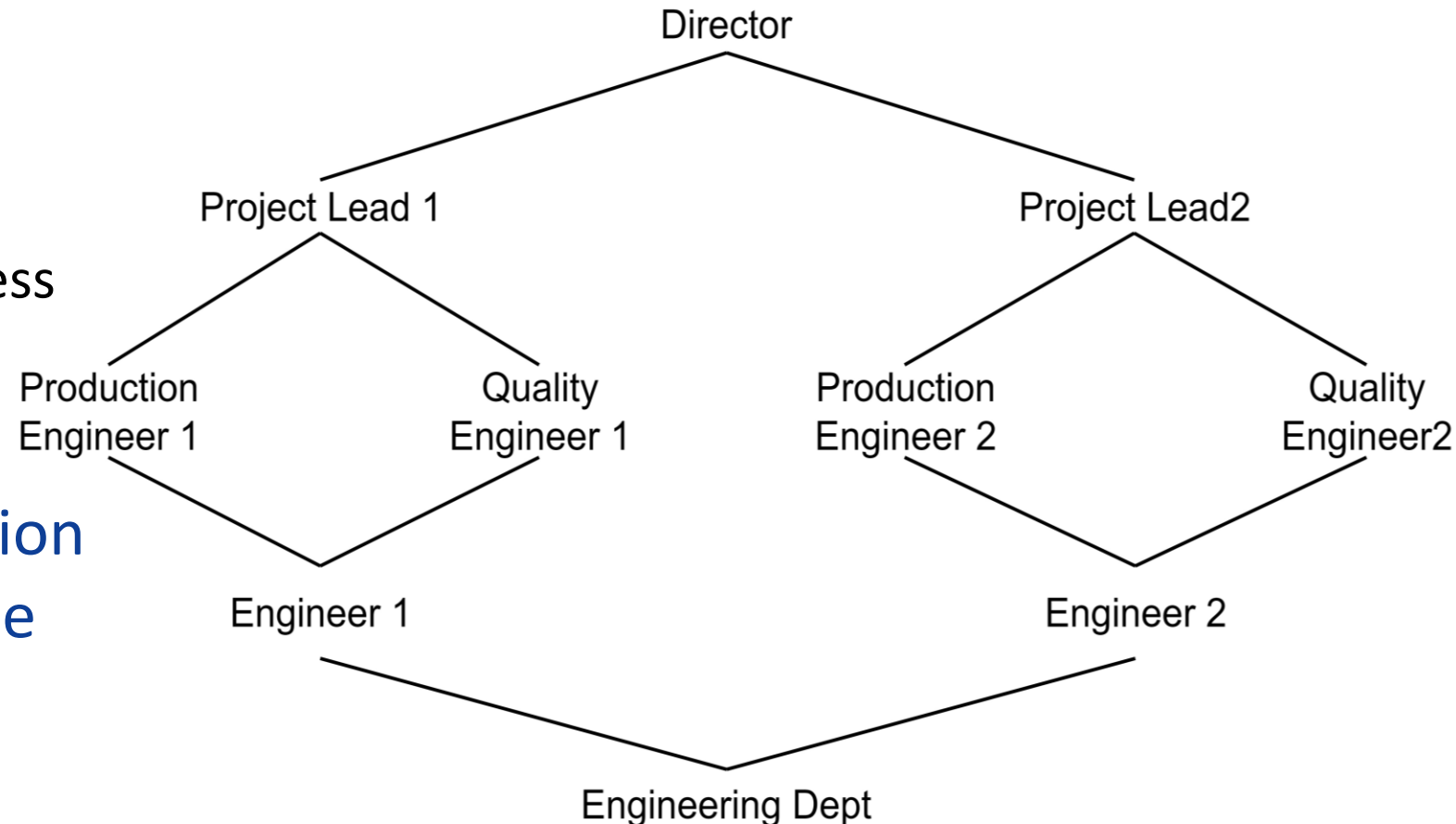
RBAC₀: Base Model

- User: an individual that has access to this computer system
 - Has an associated user ID
- Role: a named job function (authority level)
- Permission: an approval of a particular mode of access to one or more objects
- Session: a mapping between a user and set of roles to which a user is assigned



RBAC₁: Role Hierarchies

- Roles with greater responsibility
 - ▣ Greater authority to access resources
- A subordinate job function may have a subset of the access rights of the superior job function



RBAC₂: Constraints

- Adapting RBAC to the specifics of administrative and security policies in an organization

- Mutually exclusive roles

- A user can be assigned to only one role in the set (either during a session or statically)
 - Any permission (access right) can be granted to only one role in the set
 - Non-overlapping permissions, if two users are assigned to different roles in the set

- Cardinality

- Setting a maximum number of users w.r.t. roles
 - e.g., a project leader role or a department head role might be limited to a single user

- Prerequisite role

- A user can be assigned to a particular role only if it is already assigned to some other specified role
 - e.g., a user can be assigned to a senior (higher) role only if it is already assigned an immediately junior (lower) role

Attribute-based Access Control (ABAC)

- Define authorizations that express conditions on properties of both the resource and the subject
 - e.g., Alice (subject attr.) can access the HR database (resource attr.) during week days (environment attr.)
- Strength: *flexibility* and *expressive power*
- Drawback: the *performance impact* of evaluating predicates on both resource and user properties for each access
 - However, increased performance cost is less noticeable for Web services and cloud computing

ABAC Model: Attributes

● Subject attributes

- ❑ A subject is an **active** entity that causes information to flow among objects or changes the system state
- ❑ Attributes define the identity and characteristics of the subject
 - e.g., name, organization, job title

● Object attributes

- ❑ An object (or resource) is a **passive** system-related entity containing or receiving information
- ❑ Objects have attributes that can be leveraged to make access control decisions
 - e.g., file name, file size, creator

ABAC Model: Attributes (Cont.)

- Environment attributes

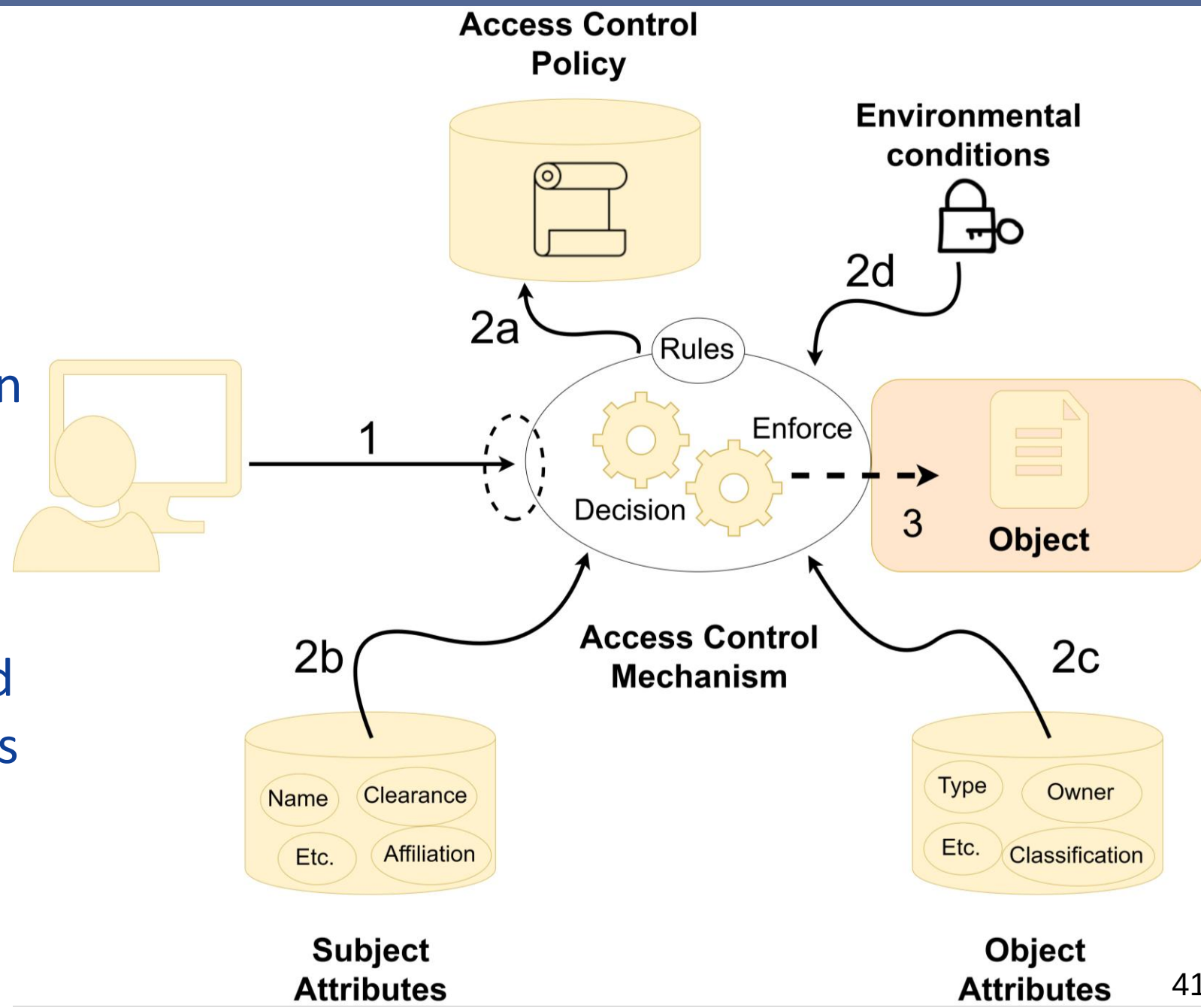
- The operational, technical, and even situational environment or context in which the information access occurs
 - e.g., current date, time, network type, etc.
- These attributes have so far been largely ignored in most access control policies

ABAC Model: Distinguishable

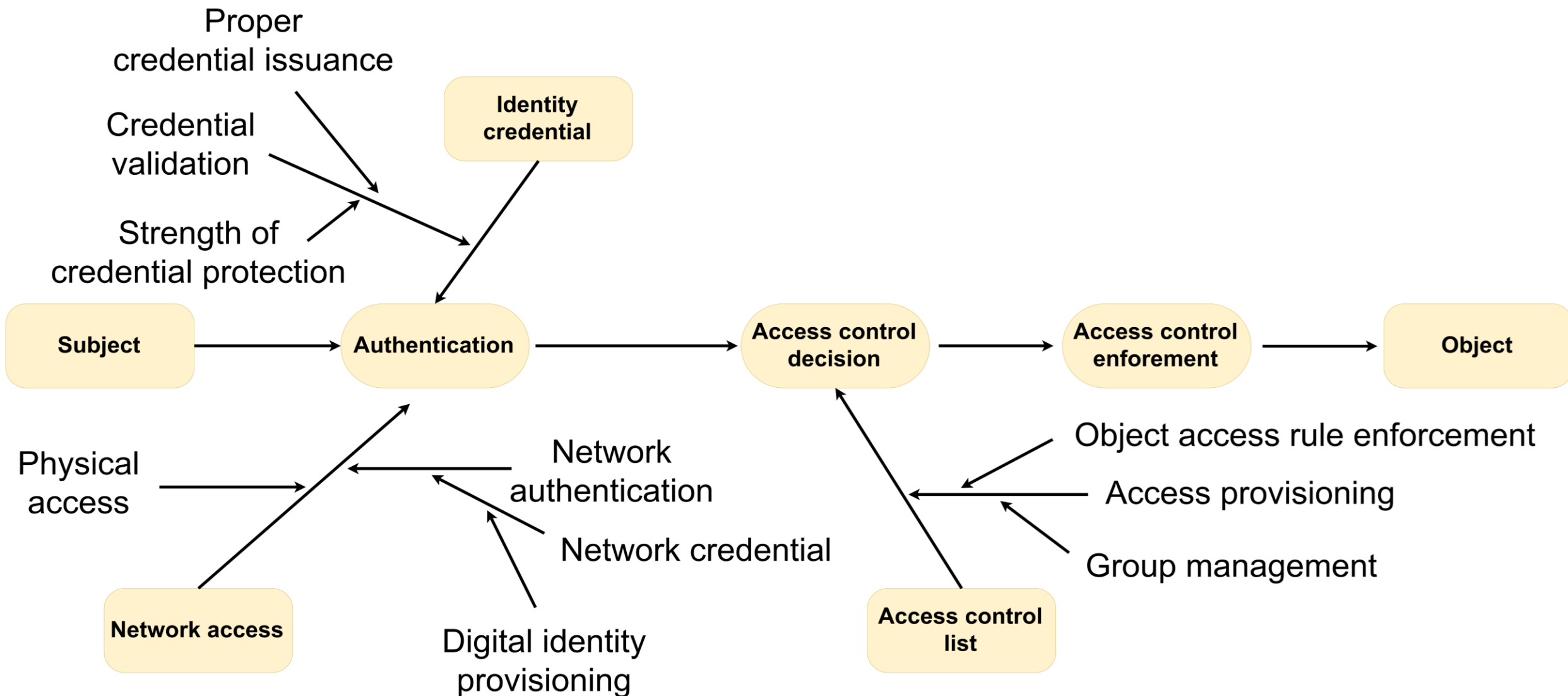
- Controls access to objects by evaluating rules against the attributes of entities (subject and object), operations, and the environment
 - Attributes may be considered characteristics of anything that may be defined
- Capable of enforcing DAC, RMAC, and MAC concepts
- Fine-grained access control: allows an unlimited number of attributes to be combined to satisfy any access control rule

ABAC Logical Architecture

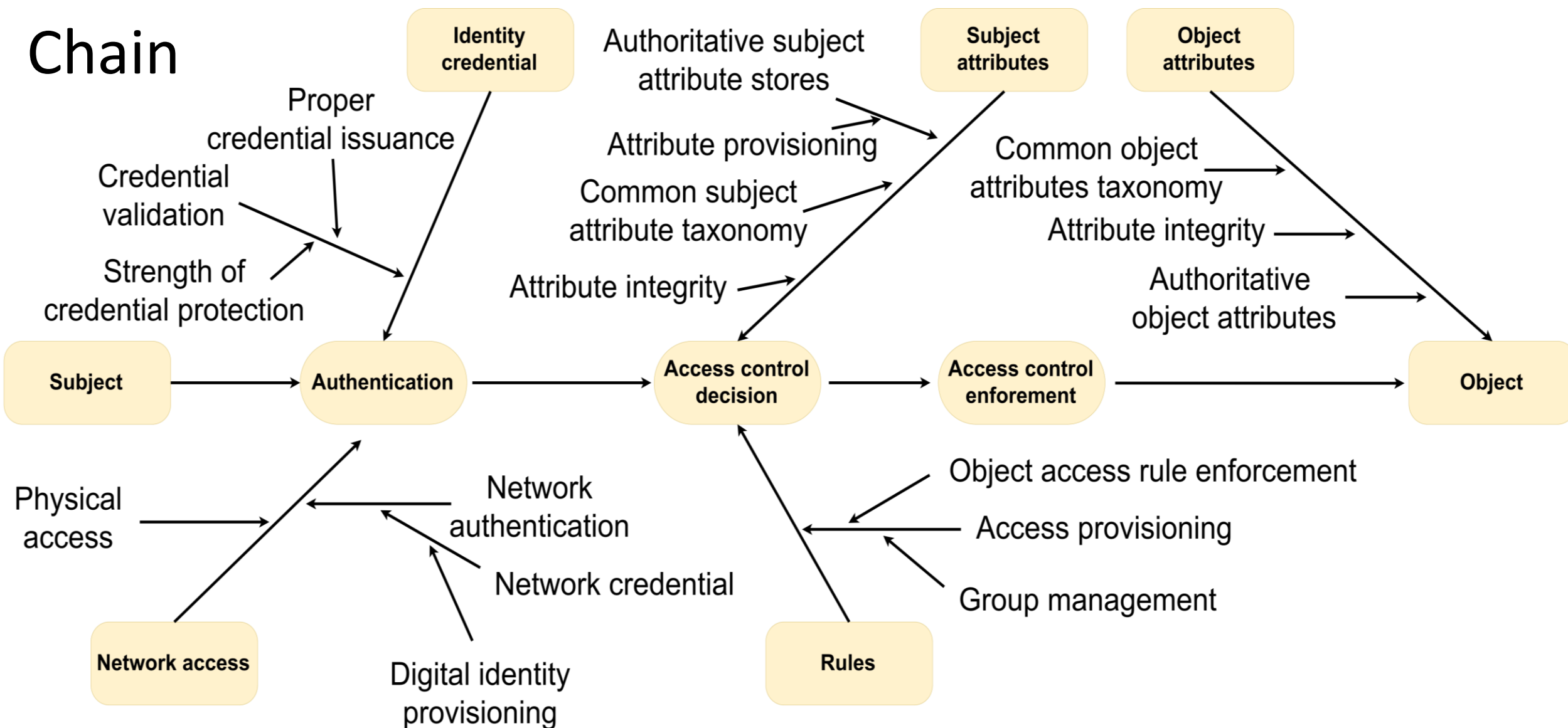
- Four independent sources of information used for the access control decision
- It is very powerful and flexible, but the cost is larger than that of other access control approaches



ACL Trust Chain



ABAC Trust Chain



ABAC Policies

- A policy is a set of rules and relationships that govern allowable behavior within an organization
 - ▣ Based on (1) privileges of subjects; (2) how resources or objects are to be protected; (3) under which environment conditions
- An ABAC policy model [YUAN05]

Subject attributes	$ATTR(s) : SA_1 \times SA_2 \times \cdots \times SA_K$
Object attributes	$ATTR(o) : OA_1 \times OA_2 \times \cdots \times OA_M$
Environment attributes	$ATTR(e) : EA_1 \times EA_2 \times \cdots \times EA_N$

Rule $can_access(s, o, e) \leftarrow f(ATTR(s), ATTR(o), ATTR(e))$

e.g., an online movie website (user u , movie m , environment e)

$can_access(u, m, e) \leftarrow (MembershipType(u) = Premium) \mid$

$(MembershipType(u) = Regular \ \& \ MovieType(m) = OldRelease \ \& \ Time(e) = 9am - 9pm)$

Case Study: RABC System for a Bank

- Dresdner bank uses a variety of computer applications over servers and mainframe computers
- In 1990, a simple DAC system was used
- For each server and mainframe computer, administrators maintained a local access control file on each host
 - ▣ Defining access rights for each employee on each application installed on the host
- However, it was cumbersome, time-consuming, and error-prone

How to solve it?

Case Study: RABC System for a Bank (Cont.)

- Dresdner bank then introduced an RBAC scheme
- The determination of access rights is compartmentalized into three different administrative units
 - ▣ Roles: a combination of official position and job function
 - ▣ Difference from NIST: a role is defined by a job function

Role	Function	Official Position
A	financial analyst	Clerk
B	financial analyst	Group Manager
C	financial analyst	Head of Division
D	financial analyst	Junior
E	financial analyst	Senior
F	financial analyst	Specialist
G	financial analyst	Assistant
...	...	...
X	share technician	Clerk
Y	support e-commerce	Junior
Z	office banking	Head of Division

Case Study: Functions and Roles for Banking

Permission Assignments

Role A: Financial analyst/Clerk

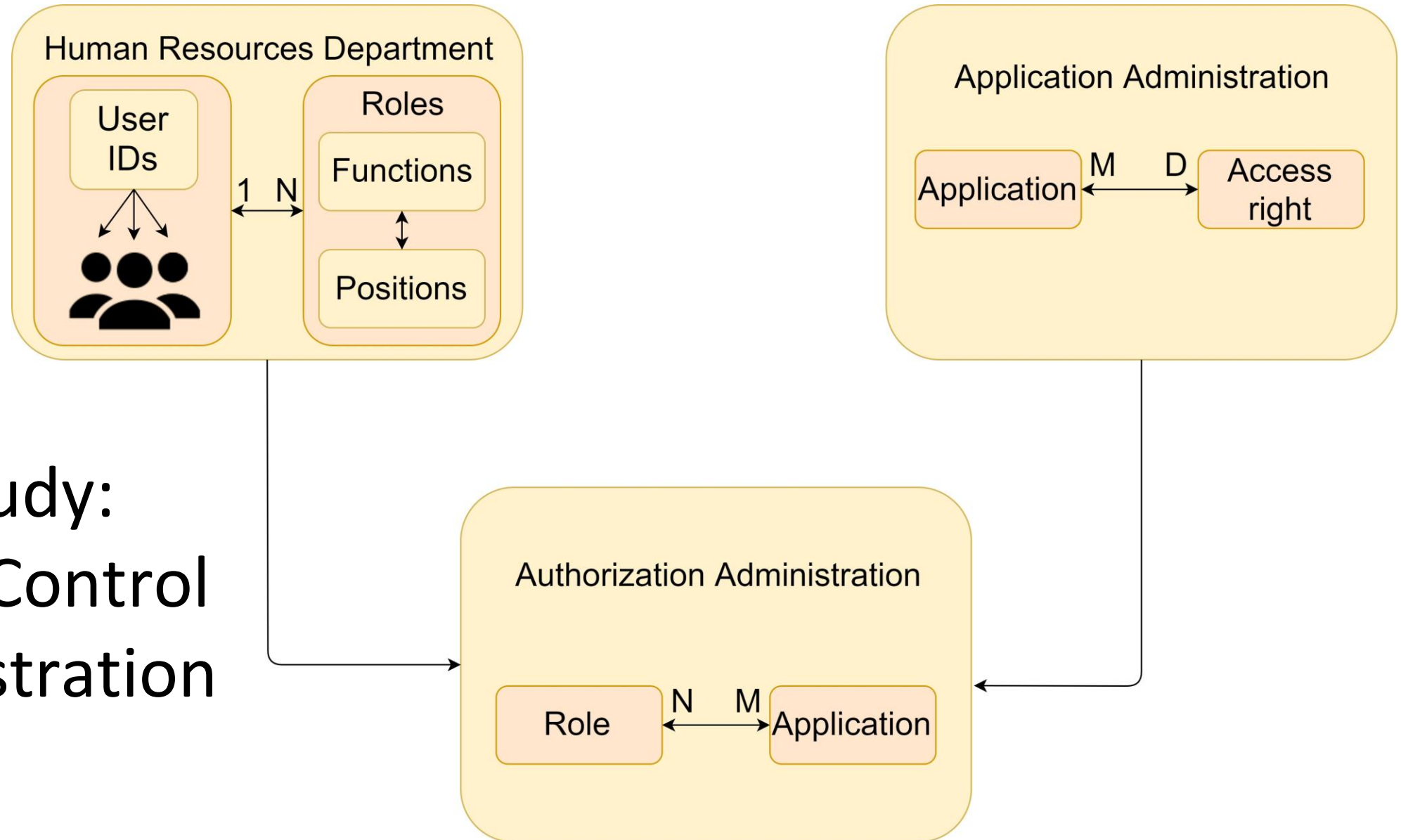
Role B: Financial analyst/Group manager

Role	Application	Access Right
A	money market instruments	1, 2, 3, 4
	derivatives trading	1, 2, 3, 7, 10, 12
	interest instruments	1, 4, 8, 12, 14, 16
B	money market instruments	1, 2, 3, 4, 7
	derivatives trading	1, 2, 3, 7, 10, 12, 14
	interest instruments	1, 4, 8, 12, 14, 16
	private consumer instruments	1, 2, 4, 7
...	...	...

Permission Assignments with Inheritance

Role	Application	Access Right
A	money market instruments	1, 2, 3, 4
	derivatives trading	1, 2, 3, 7, 10, 12
	interest instruments	1, 4, 8, 12, 14, 16
	private consumer instruments	1, 2, 4, 7
B	money market instruments	7
	derivatives trading	14
	private consumer instruments	1, 2, 4, 7
...	...	...

Case Study: Access Control Administration



Questions?